

Module 2: Analysis Of Divide & Conquer Algorithms.

★ Divide & conquer strategy:-

Divide & conquer algorithm is a problem solving strategy that involves:

- Divide: Break the problem into smaller non-overlapping problems.
- Conquer: Solve smaller problems.
- Combine: Use the solutions of smaller problems to find the overall result.

Examples of divide & conquer are Merge sort, Quick sort, Binary search etc.

Now let's look into some of its example algorithms.

★ Quick sort:-

Algorithm based on Divide & conquer that picks an element & partitions the given array around the picked pivot by placing it in a correct position in the sorted array.

There are mainly three steps in the algorithm:

1. Choose a pivot: Select an element from the array (any element) as the pivot.
2. Partition the Array: Rearrange the array around the pivot, after the partitioning

the smaller elements should be on the left & the greater elements on the right.

3. Recursive call:- Recursively apply the same process to the two partitioned subarray.

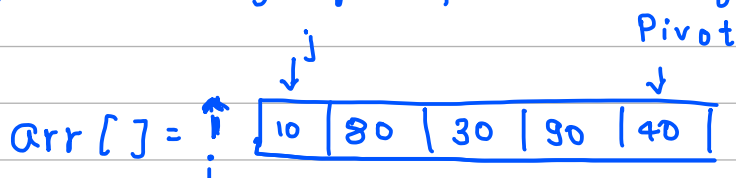
4. Base case:- The recursion stops when only 1 array is left in the sub-array.

Let's look into each of them one by one.

1) Choice of pivot:- The choice of pivot can be done in the following ways:

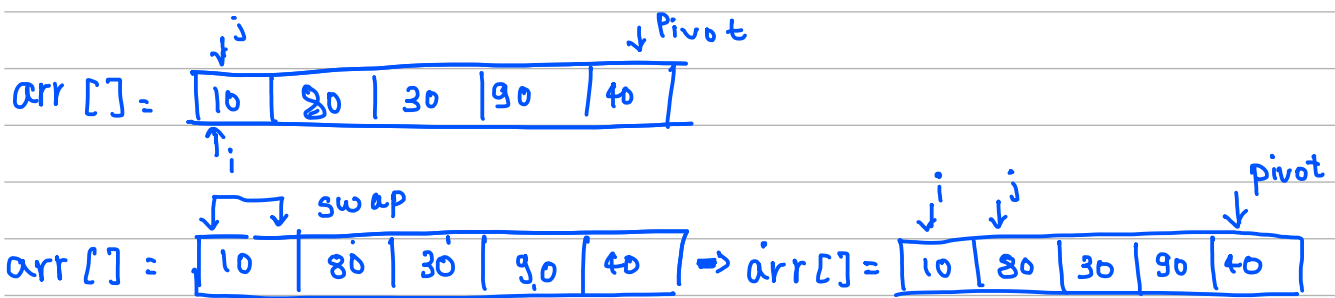
- Picking the first or the last element as pivot. The problem with this approach is that it ends up in the worst case when the array is already sorted.
- Pick a random element as a pivot. This is the preferred approach because it does not have a pattern for which the worst case happens.
- Pick the median element as pivot. This is the ideal approach in terms of time complexity, but takes more time on average as median finding has its own constraints.

Eg. :- Working of a partition algorithm :

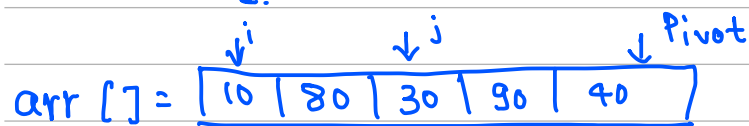


Step 1: The last element is chosen as pivot. Initialise pointer $i = -1$ & $j = 0$.

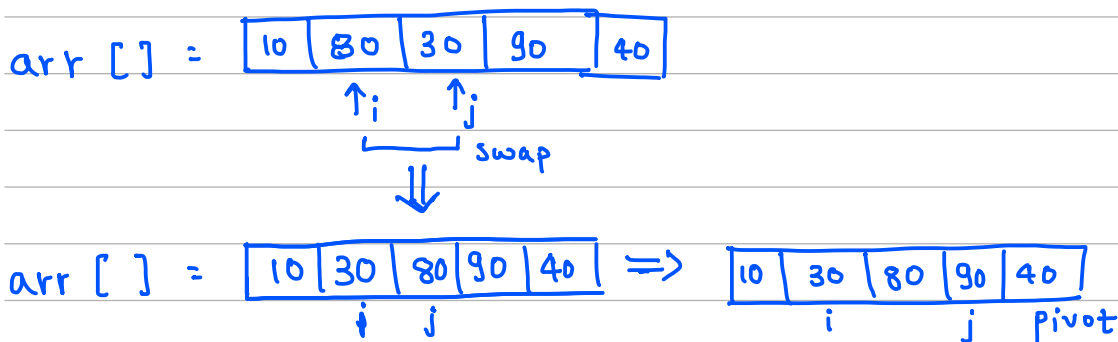
Step 2: Since $\text{arr}[i] < \text{pivot}$, increment i & swap $\text{arr}[i]$ with $\text{arr}[j]$ then increment j by 1.



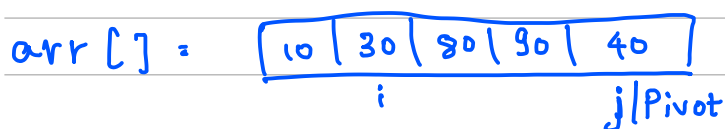
Step 3: Since $\text{arr}[j] > \text{pivot}$, No swap needed. Increment j by 1.



Step 4: Since $\text{arr}[j] < \text{Pivot}$, increment i by 1, swap $\text{arr}[i]$ & $\text{arr}[j]$, then increment j by 1.

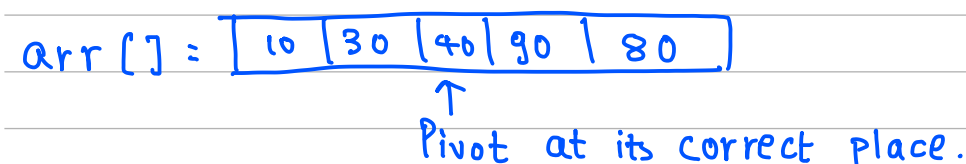


Step 5: Since $\text{arr}[j] > \text{pivot}$. No swap needed. Increment j by 1.



Step 6: Since traversal of j is over. Now move the pivot element to its correct position ($i+1$).

$\therefore$ Swap ($\text{arr}[i+1]$, $\text{arr}[4]$).

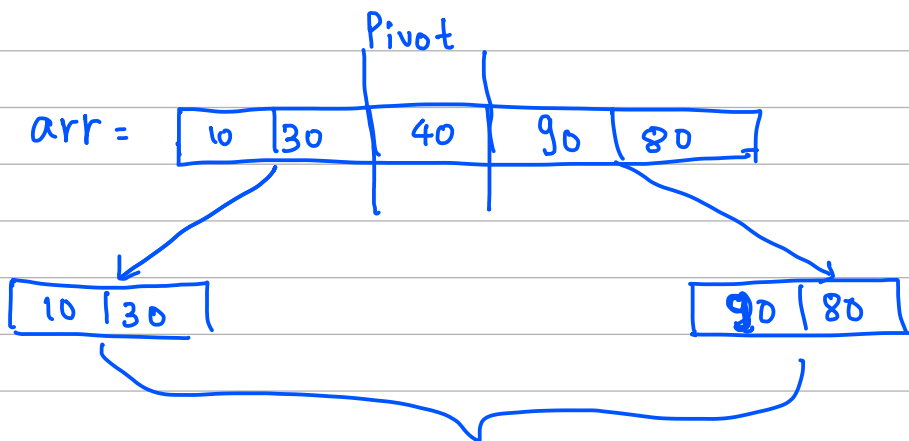


As we can see all the elements before the pivot are smaller than itself & all the elements after the pivot are greater.

Now looking at the quick sort algorithm.

We know the partitioning process rearranges the array based on the chosen pivot.

Next apply this recursively to the smaller sub arrays on the left & right sub array. Each time new pivots are selected & the arrays are again partitioned. Once every element is in its correct position the array is sorted.



Recursively call quick sort on these.

```
// Partition Algo
int partition(int arr[], int low, int high){
    // Pivot being the last element
    int pivot = arr[high];

    int i = low-1;

    // All smaller elements on the left side, whereas all the greater
    // ones on the right side of the correct position of the pivot
    for(int j=low; j<=high-1; j++){
        if(arr[j]<pivot){
            i++;
            swap(arr[i], arr[j]);
        }
    }
    // Moving the pivot to its correct position
    swap(arr[i+1], arr[high]);
    return i+1;
}
```



```
// Quick Sort Algo
void quickSort(int arr[], int low, int high){
    if(low<high){ //Base case
        // pi is the correct partition index of the pivot
        int pi = partition(arr,low,high);

        quickSort(arr,low,pi-1); // Left Subarray
        quickSort(arr, pi+1, high); // Right Subarray
    }
}
```

Complexity analysis :-

- Best case $\Omega(n \log N)$, occurs when the pivot element divides the array into two equal halves.
- Average case $\Theta(n \log N)$, The pivot divides the array into two parts but not necessarily equal
- Worst case $O(n^2)$, occurs when the smallest or largest element is always chosen as the pivot. (eg. sorted arrays).

→ Applications

- Used for general purpose in memory sorting due to its fast average case performance.
- Used in many standard library implementations like C++ standard template library (STL as part of hybrid algos).
- Useful when cache efficiency & practical & important in real time applications.

★ Merge Sort:-

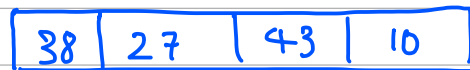
Merge sort is a sorting algorithm that follows the Divide & Conquer approach. It works by recursively dividing the input array into two halves, recursively

sorting the two halves & finally merging them back together to obtain the sorted array.

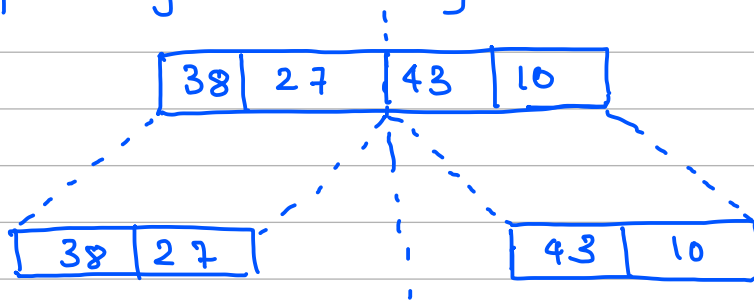
It works on 3 basic steps :

- 1) Divide : Divide the list or array recursively into two halves until it can no more be divided.
- 2) Conquer : Each subarray is sorted individually using the merge sort algo.
- 3) Merge : The sorted subarrays are merged back together in sorted order. This continues until all the elements from both arrays are merged.

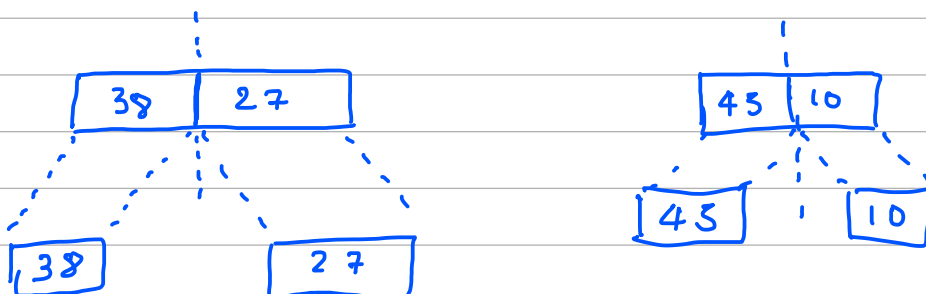
Eg. let's sort the array or list [38, 27, 43, 10].



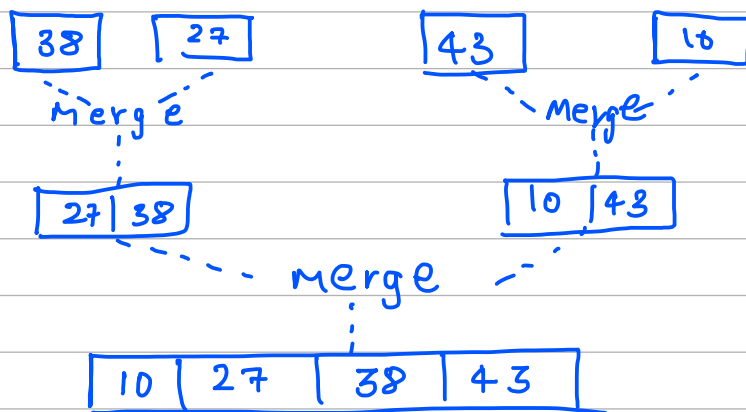
step 01: Splitting the Array into two equal halves.



step 02: Splitting the subarrays into two halves.



Step 3: Merging unit length cells into sorted subarrays.



Code implementation :-

```
void merge(int arr[], int left, int mid, int right){
    int n1 = mid-left+1;
    int n2 = right -mid;
    // Create temp vectors
    int l[n1], r[n2];
    for(int i =0;i<n1;i++){
        l[i] = arr[left+i];
    }
    for(int i=0;i<n2;i++){
        r[i] = arr[mid+1+i];
    }
    int i = 0, j =0;
    int k = left;
    // Merge the temp vectors back into arr[left...right]
    while(i<n1 && j<n2){
        if(l[i]<=r[j]){
            arr[k] = l[i];
            i++;
        }else{
            arr[k] = r[j];
            j++;
        }
        k++;
    }
    // Copy the remaining elements of l[], if there are any
    while(i<n1){
        arr[k] = l[i];
        i++;
        k++;
    }
    // Copy the remaining elements of r[], if there are any
    while (j<n2){
        arr[k] = r[j];
        j++;
        k++;
    }
}
```

```

void mergeSort(int arr[], int left, int right){
    if(left >= right){
        return;
    }
    int mid = left + (right-left)/2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid+1, right);
    merge(arr, left, mid, right);
}

int main(){
    int arr[] = {38, 27, 43, 10};
    int n = 4; //Array size
    mergeSort(arr, 0, n-1);
    for(int i:arr) cout<<i<<" ";
    cout<<endl;
}

```

Complexity analysis of Merge sort using recurrence relation.

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

↑
↑
 Because n comparisons each step.
 the array gets split into two

Now with the same logic

$$\begin{aligned}
 T_n &= 2 \left[2T\left(\frac{n}{4}\right) + \frac{n}{2} \right] + n \\
 &= 4T\left(\frac{n}{4}\right) + n + n \\
 &= 4T\left(\frac{n}{4}\right) + 2n
 \end{aligned}$$

again

$$\begin{aligned}
 T_n &= 4 \left[2T\left(\frac{n}{8}\right) + \frac{n}{4} \right] + 2n \\
 &= 8T\left(\frac{n}{8}\right) + 3n
 \end{aligned}$$

This can be generalised as:-

$$T_n = 2^k \cdot T\left(\frac{n}{2^k}\right) + k n$$

$$\text{let } 2^k = n$$

$$k \log_2 2 = \log_2 n$$

$$k = \log_2 n$$

$$\therefore T_n = n T\left(\frac{2^k}{2^k}\right) + n \log_2 n$$

$$= n T(1) + n \log n$$

$$= n + n \log n.$$

The dominating term here is $n \log n$.

$\therefore$ The T.C becomes $O(n \log n)$.

This same rule is applied for all the cases, the best, worst & average.

The space complexity of Merge sort is $O(n)$

★ Recurrence Relation:-

A recurrence relation is a mathematical expression that defines a sequence in terms of its previous terms. Often used to model the time complexity of recursive algorithms.

Eg 1)

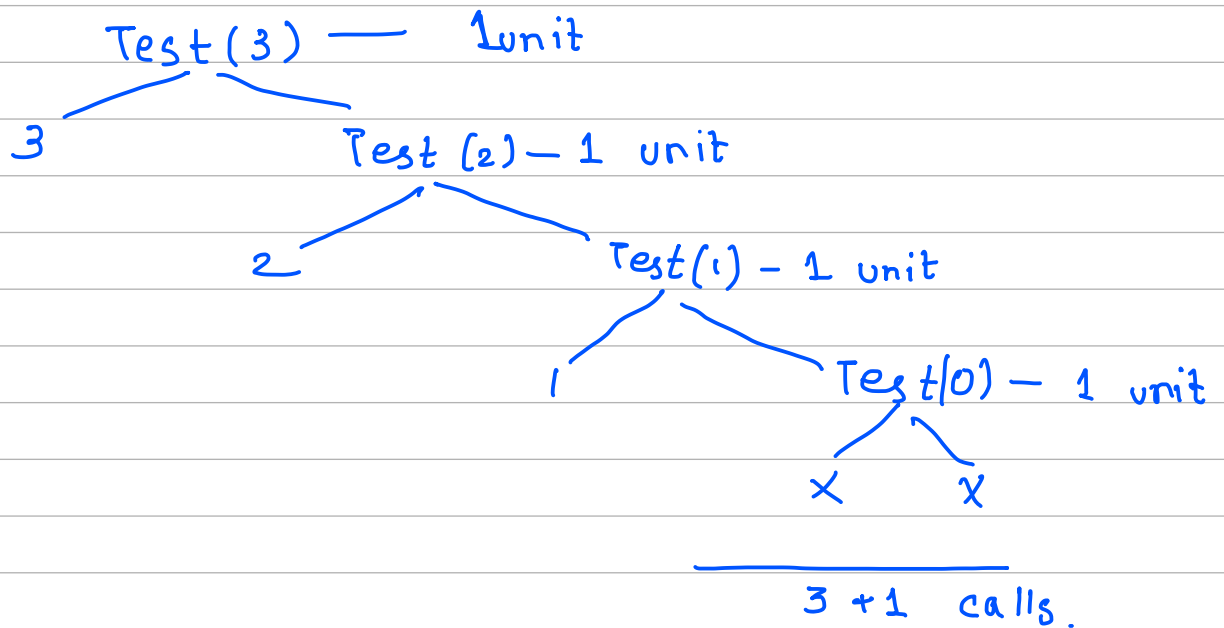
```
void Test(int n) {  
    if (n > 0) {  
        printf("%d", n);    — 1 unit of time  
    }  
}
```

```

Test(n-1);
}

```

making a recursion tree for $n=3$.



$$\Rightarrow T(n) = n + 1 \text{ calls}$$

$$\Rightarrow O(n).$$

By substitution method:-

$$T(n) = \begin{cases} 1 & n=0 \\ T(n-1) + 1 & n > 0 \end{cases}$$

$$T(n) = T(n-1) + 1$$

further solving for $T(n-1)$

$$T(n) = [T(n-2) + 1] + 1$$

$$T(n) = T(n-2) + 2$$

Again solving further.

$$T(n) = T(n-3) + 3$$

⋮ till k steps.
⋮

$$T(n) = T(n-k) + k$$

let $k = n$

$$T(n) = T(0) + n$$

$$= n + 1$$

$$\approx O(n)$$

This is the same relation we got from the recursion tree.

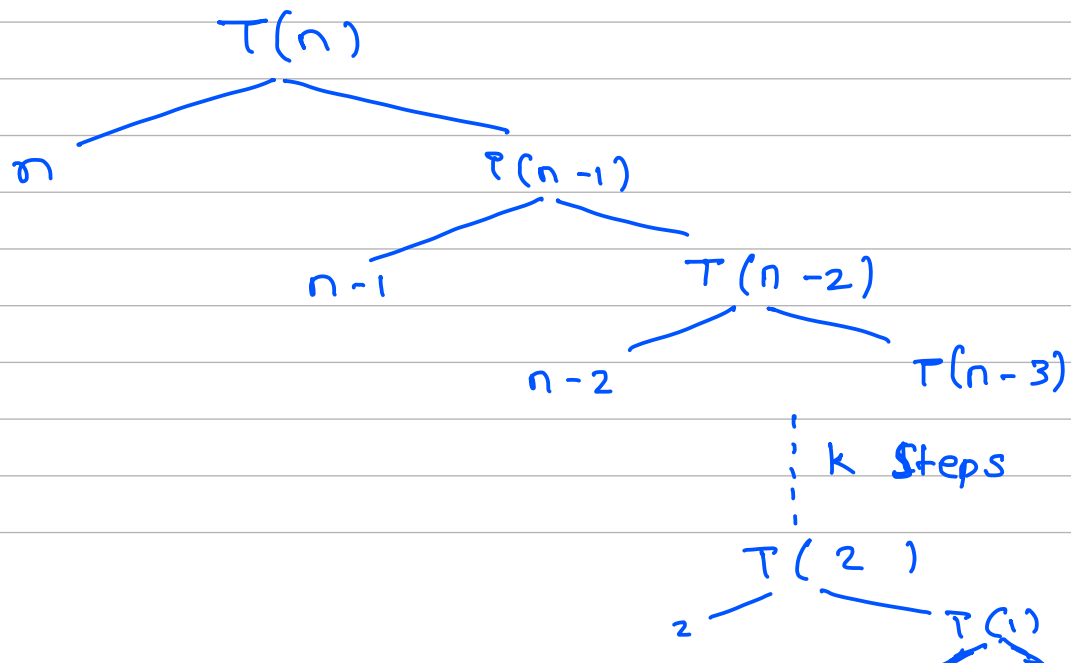
Eg. 2.

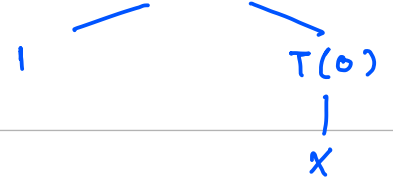
```
void test(int n) {  $\rightarrow T(n)$ 
  if (n > 0) {
    for (i=0; i<n; i++) {
      printf("%d ", n);
    }
    test(n-1);  $\rightarrow T(n-1)$ 
  }
}
```

$T(n) = T(n-1) + n$

Using recursion tree:-

$$T(n) = \begin{cases} 1 & n = 0 \\ T(n-1) + n & n > 0 \end{cases}$$





$$\Rightarrow T(n) = 0 + 1 + 2 + 3 + \dots + n-2 + n-1 + n = \frac{n(n+1)}{2}$$

$$= O(n^2)$$

Using substitution method:-

$$T(n) = T(n-1) + n$$

Solving for $T(n-1)$

$$T(n) = [T(n-2) + n-1] + n$$

$$= T(n-2) + (n-1) + n \quad \text{--- (1)}$$

Again solving for $T(n-2)$

$$T(n) = [T(n-3) + n-2] + (n-1) + n$$

$$T(n) = T(n-3) + (n-2) + (n-1) + n \quad \text{--- (2)}$$

⋮
k times

$$T(n) = T(n-k) + (n-(k-1)) + (n-(k-2)) + \dots + (n-1) + n$$

Let $n = k$

$$T(n) = T(0) + [1 + 2 + 3 + \dots + (k-1) + k]$$

$$= T(0) + \left[\frac{k(k+1)}{2} \right]$$

$$\therefore n = k$$

$$T(n) = T(1) + \frac{n(n+1)}{2}$$

$$= 1 + \frac{n(n+1)}{2}$$

$$= O(N^2)$$

★ Master's Theorem for dividing functions:-

General form :- $T(n) = aT(n/b) + f(n)$

where $a > 1$, $b > 1$ & $f(n) = O(n^k \log^p n)$

from this we find

$$\textcircled{i} \log_b a$$

$$\textcircled{ii} k$$

Case 1:- $\log_b a > k$ then $O(n^{\log_b a})$

Case 2:- $\log_b a = k$

$$2.1 \text{ if } p > -1 \quad O(n^k \log^{p+1} n).$$

$$2.2 \text{ if } p = -1 \quad O(n^k \log \log n).$$

$$2.3 \text{ if } p < -1 \quad O(n^k)$$

Case 3:- $\log_b a < k$

$$3.1 \text{ if } p \geq 0 \quad O(n^k \log^p n)$$

$$3.2 \quad p < 0 \quad O(n^k)$$

Eg 1.

$$T(n) = 2T(n/2) + 1$$

$$a = 2$$

$$b = 2$$

$$f(n) = 1$$

$$\Rightarrow n^k \log^p n = 1$$

$$k = 0, p = 0$$

$$\text{here } \log_b a = \log_2 2 = 1 \quad \& \quad k = 0$$

$\therefore \log_b a > k$ it is case one

$\therefore$ The t.c becomes $O(n^{\log_b a})$

$$= O(n^{\log_2 2})$$

$$= O(n)$$

Eg. 2

$$T(n) = 2T\left(\frac{n}{4}\right) + \sqrt{n}$$

$$a = 2$$

$$b = 4$$

$$f(n) = \sqrt{n}$$

$$n^k \log^p n = n^{1/2}$$

$$k = 1/2, p = 0$$

$$\log_b a = \log_4 2 = 1/2$$

Here $\log_b a = k$ this follows case 2 & since $p < -1$
the t.c becomes $O(n^k \log^{p+1} n)$

$$= O(n^{1/2} \log n) = O(\sqrt{n} \log n).$$

eg. 3

$$T(n) = 4T\left(\frac{n}{2}\right) + n^2 \log n.$$

$$a = 4$$

$$b = 2$$

$$f(n) = n^2 \log n$$

$$n^k \log^p n$$

$$k=2, p=1$$

$$\log_b a = \log_2 4 = 2$$

here $\log_b a = k$ which follows case 2 & since $p > -1$
the t.c becomes :- $O(n^k \log^{p+1} n)$

$$= O(n^2 \log^{1+1} n) = O(n^2 \log^2 n).$$